



Métodos e Algoritmos para Otimização Multiobjetivo

ga – MATLAB

Lino Costa – lac@dps.uminho.pt

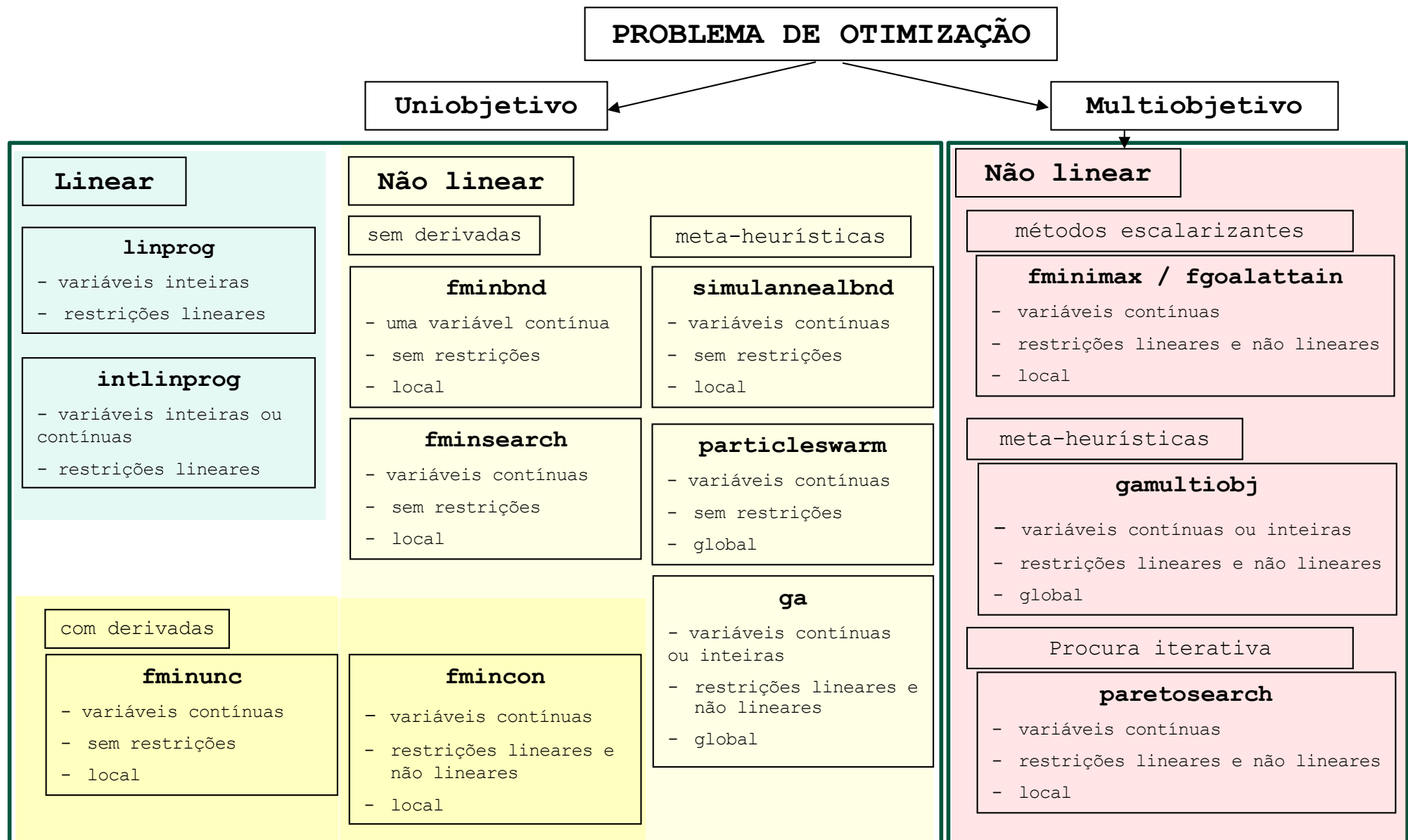
Departamento de Produção e Sistemas

Ano letivo 2025/2026



Visão geral (solvers do MatLab)...

Universidade do Minho





Global Optimization Toolbox

Universidade do Minho

<code>ga</code>	Find minimum of function using genetic algorithm
<code>gamultiobj</code>	Find Pareto front of multiple fitness functions using genetic algorithm

Outline of the Algorithm

The following outline summarizes how the genetic algorithm works:

1. The algorithm begins by creating a random initial population.
2. The algorithm then creates a sequence of new populations. At each step, the algorithm uses the individuals in the current generation to create the next population. To create the new population, the algorithm performs the following steps:
 - a. Scores each member of the current population by computing its fitness value. These values are called the raw fitness scores.
 - b. Scales the raw fitness scores to convert them into a more usable range of values. These scaled values are called expectation values.
 - c. Selects members, called parents, based on their expectation.
 - d. Some of the individuals in the current population that have lower fitness are chosen as *elite*. These elite individuals are passed to the next population.
 - e. Produces children from the parents. Children are produced either by making random changes to a single parent—*mutation*—or by combining the vector entries of a pair of parents—*crossover*.
 - f. Replaces the current population with the children to form the next generation.
3. The algorithm stops when one of the stopping criteria is met. See [Stopping Conditions for the Algorithm](#).
4. The algorithm takes modified steps for linear and integer constraints. See [Integer and Linear Constraints](#).
5. The algorithm is further modified for nonlinear constraints. See [Nonlinear Constraint Solver Algorithms](#).

References

[1] Goldberg, David E., *Genetic Algorithms in Search, Optimization & Machine Learning*, Addison-Wesley, 1989.



Execução do `ga` com as opções por omissão

Universidade do Minho

Executar o comando `ga` com a sintaxe:

```
[x,fval] = ga(@fitnessfun, nvars)
```

Os argumentos de entrada são

- `@fitnessfun` — a função definida numa M-file com a função objetivo (de forma idêntica ao que se faz com os comandos `fminunc` da Optimization toolbox)
- `nvars` — o número de variáveis da função objetivo.

Os argumentos de saída são:

- `x` — a melhor aproximação para o minimizante obtida pelo algoritmo
- `fval` — o valor da função objetivo no ponto `x` (aproximação ao mínimo da função objetivo)

O comando `ga` possui mais argumentos. Podem ser consultados usando a ajuda:

```
help ga
```

ou consultando a documentação:

```
doc ga
```



Comando ga

Universidade do Minho

MatLab

Encontrar o mínimo da função Rastrigins com todos as opções de procura por defeito.

1º Definir uma M-file `rastrigins.m` com a função $f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos(2\pi x_1) + \cos(2\pi x_2))$

```
function f = rastrigins(x)
f = 20+x(1)^2 + x(2)^2 - 10*(cos(2*pi*x(1))+cos(2*pi*x(2)));
end
```

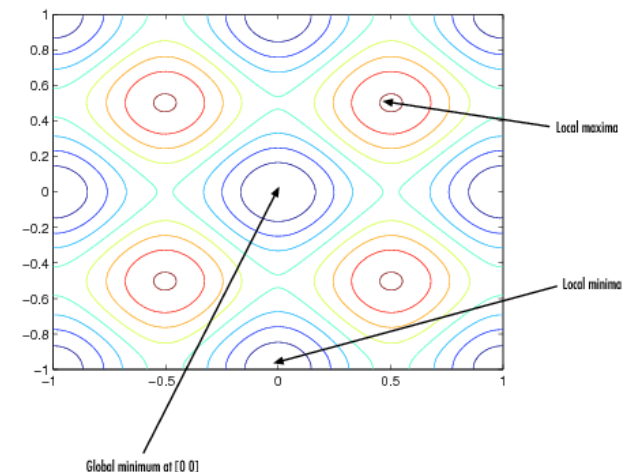
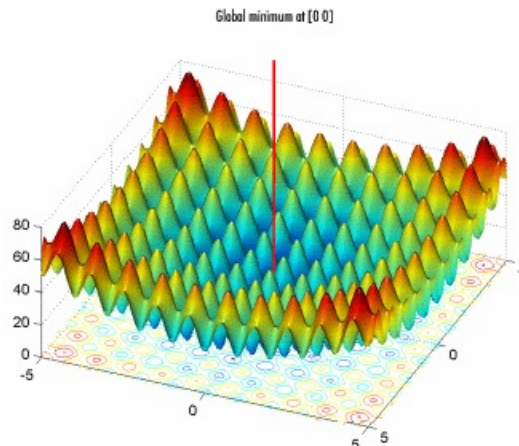
2º Executar o comando:

```
[x fval] = ga(@rastrigins, 2)
```

Devolve...

```
x =
    0.0027    -0.0052

fval =
    0.0068
```





Argumentos de saída do comando `ga`

Universidade do Minho

Para obter mais informação acerca da execução, fazer:

```
[x,fval,exitflag,output,population,scores] = ga(@fitnessfcn, nvars)
```

Argumentos de saída:

`x` - minimizante

`fval` – valor da função objetivo no minimizante

`exitflag` – valor inteiro correspondente à razão pela qual o algoritmo terminou

`output` – estrutura contendo informação acerca do processo de otimização

`population` – população final

`scores` – medidas de aptidão da população final

MatLab

Executar o comando para obter mais informação da procura:

```
[x,fval,exitflag,output,population,scores] = ga (@rastrigins, 2)
```



Opções do comando `ga`

Universidade do Minho

Pode-se especificar opções para o comando `ga` pela passagem de um estrutura de opções:

```
[x,fval] = ga(@fitnessfun, nvars, [],[],[],[],[],[],[],options)
```

Nota: no comando anterior não são especificadas nenhuma restrições lineares de igualdade, lineares de desigualdade ou não lineares.

MatLab

Pode-se consultar/alterar a estrutura de opções utilizando o comando `optimoptions`.

Para consultar todas as opções disponíveis (e os valores por defeito), fazer:

```
optimoptions
```

Para consultar as opções por defeito e colocá-las em `options`, fazer:

```
options = optimoptions(@ga)
```

Para criar uma estrutura de opções com a opção `'PopulationSize'` igual a 50, fazer

```
options = optimoptions(@ga,'PopulationSize', 50)
```



Comando `optimoptions`

Universidade do Minho

```
PopulationType: [ 'bitstring'      | 'custom'      | {'doubleVector'} ]  
PopInitRange:  [ matrix           | {[ -10;10]} ]  
PopulationSize: [ positive scalar ]  
EliteCount:    [ positive scalar | {0.05*PopulationSize} ]  
CrossoverFraction: [ positive scalar | {0.8} ]
```

```
ParetoFraction: [ positive scalar | {0.35} ]
```

```
MigrationDirection: [ 'both'      | {'forward'} ]
```

```
MigrationInterval: [ positive scalar | {20} ]
```

```
MigrationFraction: [ positive scalar | {0.2} ]
```

- Tipo de indivíduo
- Gama de valores para geração da população inicial (caso não tenha restrições de limite simples)
- Tamanho da População
- Elitismo
- Fração de Cruzamento

```
Generations: [ positive scalar ]  
TimeLimit:    [ positive scalar | {Inf} ]  
FitnessLimit: [ scalar          | {-Inf} ]  
StallGenLimit: [ positive scalar ]  
StallTest:    [ 'geometricWeighted' | {'averageChange'} ]  
StallTimeLimit: [ positive scalar | {Inf} ]  
TolFun:       [ positive scalar ]
```

```
TolCon: [ positive scalar | {1e-6} ] ...
```

- Nº máximo de gerações
- Tempo limite
- Menor valor da f. obj.
- Nº de gerações sem decréscimo médio da f. obj. menor que tolerância
- Tempo máximo sem decréscimo da f. obj.
- Tolerância no valor da f. obj.



Indivíduos ou cromossomas - PopulationType

Universidade do Minho

Problemas com variáveis contínuas ($x_1, \dots, x_n$)

Código binário padrão / código Gray

x_1	x_2	...	x_n
0 1 0 1 1 0 1 1	0 1 0 0 1 0 0 1	...	1 0 0 1 1 0 0 0

'bitstring'

Problemas com variáveis inteiras ($y_1, \dots, y_s$)

Código binário padrão / código Gray

y_1	y_2	...	y_s
0 1 0 1 1 0 1 1	0 1 0 0 1 0 0 1	...	1 0 0 1 1 0 0 0

ou

Vetor de números reais

x_1	x_2	...	x_n
2.340	-1.062	...	7.464

'doubleVector'

ou

Vetor de números inteiros

y_1	y_2	...	y_s
4	5	...	-3

Argumento 'intcon'

Problemas mistos ($x_1, \dots, x_n, y_1, \dots, y_s$)

Combinação de representações
(exemplo vetor de números reais e inteiros)

x_1	x_2	...	x_n	y_1	y_2	...	y_s
2.340	-1.062	...	7.464	4	5	...	-3
'doubleVector'				Argumento 'intcon'			

Problemas combinatórios ($z_1, \dots, z_w$)

Permutação (exemplo para $w = 5$)

z_1	z_2	z_3	z_4	z_5
'custom'				



Configuração do ga

Universidade do Minho

`'PopulationType'` – especifica o tipo de representação dos indivíduos da população

- `'doubleVector'` — representação por vetores de número reais (valor por defeito)
- `'bitstring'` — representação por sequências de dígitos binários.

NOTA: A M-file com função objetivo tem que ser definida para valores binários, i.e., têm de ser implementadas as conversões binário e Gray para número inteiro ou número real.

- `'custom'` — para definir uma outra representação que não as anteriores

NOTA: Neste caso é preciso definir as funções de criação, cruzamento e mutação.

`'PopInitRange'` – especifica a gama de valores por variável utilizada na criação da população inicial

NOTA: é uma matriz com duas colunas e um número de linhas igual ao número de variáveis (com a forma `[lb;ub]`). Só é usado caso não tenha restrições de limite simples.

`'PopulationSize'` – especifica o número de indivíduos na população

`'EliteCount'` – especifica o número de indivíduos que constituem a elite

`'CrossoverFraction'` – especifica a proporção de indivíduos a que é aplicado o crossover (exclui a elite)

MatLab

Para gerar a população inicial com valores entre -5 e 5 para as duas variáveis, fazer

```
options = optimoptions(@ga,'PopulationSize', 50);  
options = optimoptions(@ga,options,'PopInitRange',[-5 5;-5 5])  
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Configuração do ga

Universidade do Minho

- 'Generations' — especifica o número máximo de gerações
- 'TimeLimit' — especifica o tempo máximo de execução em segundos
- 'FitnessLimit' — o algoritmo para se o melhor indivíduo tem um valor da função objetivo inferior a este limite
- 'StallGenLimit' — o algoritmo para se a média pesada da função objetivo ao fim deste número de gerações é inferior à tolerância definida para a função objetivo (TolFun)
- 'StallTimeLimit' — o algoritmo para se não houver uma melhoria do valor da função objetivo do melhor indivíduo ao fim deste limite de tempo
- 'TolFun' — Tolerância da função objetivo.

MatLab

Para alterar o critério de paragem por forma a que o número máximo de gerações seja 150 e que a tolerância da função objetivo seja testada com base em 100 gerações, fazer:

```
options = optimoptions(@ga, options, 'Generations', 150, 'StallGenLimit', 100);  
[x, fval, exitflag, output] = ga(@rastrigins, 2, [], [], [], [], [], [], [], [], options)
```



...comando `optimoptions`

Universidade do Minho

```
InitialPopulation: [ matrix          | {} ]  
InitialScores: [ column vector    | {} ]
```

- Indivíduos a incluir na população inicial
- Medidas de aptidão iniciais
- Método de geração da população inicial

```
NonlinConAlgorithm: [ 'penalty' | {'auglag'} ]  
InitialPenalty: [ positive scalar | {10} ]  
PenaltyFactor: [ positive scalar | {100} ]
```

```
CreationFcn: [ function handle | @gacreationuniform | @gacreationlinearfeasible ]  
FitnessScalingFcn: [ function_handle | @fitscalingshiftlinear | @fitscalingprop |  
                    @fitscalingtop | {@fitscalingrank} ]  
SelectionFcn: [ function_handle | @selectionremainder | @selectionuniform |  
               @selectionroulette | @selectiontournament | @selectionstochunif ]  
CrossoverFcn: [ function_handle | @crossoverheuristic | @crossoverintermediate |  
               @crossoveringlepoint | @crossovertwopoint | @crossoverarithmetic |  
               @crossoverscattered ]  
MutationFcn: [ function_handle | @mutationuniform | @mutationadaptfeasible |  
              @mutationgaussian ]
```

```
DistanceMeasureFcn: [ function_handle | {@distancecrowding} ]  
HybridFcn: [ @fminsearch | @patternsearch | @fminunc | @fmincon |
```

- Tipo de Medição do Desempenho
- Tipo de Seleção
- Tipo de Cruzamento
- Tipo de Mutação



Criação da população inicial - CreationFcn

Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Tendo em conta a representação adotada, P indivíduos da população inicial, podem ser gerados.

Exemplos de mecanismo de criação da população inicial:

- completamente aleatória
 - Código binário: cada gene (0 ou 1) é gerado de acordo com uma distribuição uniforme discreta de probabilidade 1/2
 - Número real: cada gene é gerado de acordo com uma distribuição uniforme contínua entre a e b (finitos)
 - Número inteiro: cada gene (k valores inteiros) é gerado de acordo com uma distribuição uniforme discreta entre a e b (finitos)
- @gacreationuniform
- aleatória, satisfazendo restrições de limite simples
- aleatória, satisfazendo restrições de limite simples e restrições lineares
- @gacreationlinearfeasible
- ...



Configuração do ga

Universidade do Minho

`'InitialPopulation'` – especifica uma população inicial

`'InitialScores'` – especifica os valores da função de aptidão para a população inicial

`'CreationFcn'` – especifica a função de criação da população inicial:

- `@gacreationuniform` – cria uma população inicial de acordo com uma distribuição uniforme (valor por defeito se não existirem restrições lineares)
- `@gacreationlinearfeasible` – cria uma população inicial que satisfaz as restrições de limite e lineares (valor por defeito se existirem restrições lineares)
- `Custom` – permite definir uma função de criação da população

MatLab

Para incluir o ponto [2 2] na população inicial, fazer

```
options = optimoptions(@ga, options, 'InitialPopulation',[2 2]);  
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Configuração do ga

Universidade do Minho

`'FitnessScalingFcn'` – especifica o tipo de função de avaliação (fitness function)

- `@fitscalingrank` — ranking linear com $1/\sqrt{\text{Rank}}$ (o valor por defeito).
- `@fitscalingprop` — proporcional.
- `@fitscalingtop` — a apenas um determinado número dos melhores indivíduos é atribuído um mesmo valor de fitness diferente de zero; aos restantes é atribuído zero.
- `@fitscalingshiftlinear` — escala linear.
- `Custom` — permite definir a função de avaliação.

MatLab

Para passar a utilizar a função de avaliação proporcional, fazer

```
options = optimoptions(@ga, options, 'FitnessScalingFcn', @fitscalingprop);  
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Avaliação dos indivíduos - FitnessScalingFcn

Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Devido à utilização de esquemas de seleção parental probabilísticos, os indivíduos devem ser avaliados utilizando uma função de aptidão (*fitness function*) que toma apenas valores reais positivos, isto é,

$$F(\mathbf{x}): \mathbb{R} \rightarrow \mathbb{R}^+$$

Exemplos de funções de aptidão:

- Graduação (*ranking scaling*)
`@fitscalingrank`
em que o melhor indivíduo tem *rank* 1 e o pior tem *rank* *P*
- Desvio em relação ao melhor (*Shift bestfit scaling*)
`@fitscalingprop`
- Escala linear (*Linear scaling*)
`@fitscalingshiftlinear`
sendo *a* e *b* calculados de tal forma que $F(\mathbf{x}) > 0$
- ...



Configuração do ga

Universidade do Minho

`'SelectionFcn'` – especifica o tipo de seleção

- `@selectionstochunif` — seleção estocástica uniforme (valor por defeito).
- `@selectionremainder` — remainder selection.
- `@selectionuniform` — seleção uniforme.
- `@selectionroulette` — seleção por roleta.
- `@selectiontournament` — seleção por torneio (por defeito, 4 indivíduos)
- `Custom` – permite definir o mecanismo de seleção.

MatLab

Para passar a utilizar a seleção por torneio com 2 indivíduos, fazer

```
options = optimoptions(@ga, options, 'SelectionFcn', {@selectiontournament,2});  
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],[],options)
```



Seleção parental - SelectionFcn

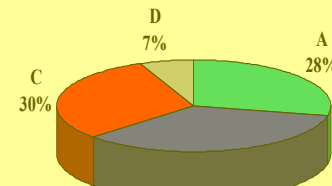
Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. **Seleção parental**
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Seleção dos progenitores para gerar descendentes para a geração seguinte.

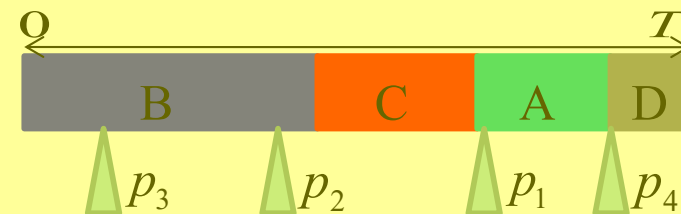
Exemplos de mecanismos de seleção parental:

- **Roleta (*roulette wheel*)** – simula uma roleta com secções de área proporcional à aptidão dos indivíduos da população. Números aleatórios são gerados para seleccionar os progenitores.



$$T = \sum_{i=1}^P F(\mathbf{x})$$

@selectionroulette



$$p_i \sim U(0, T) \quad i = 1, \dots, P$$



Seleção parental - SelectionFcn

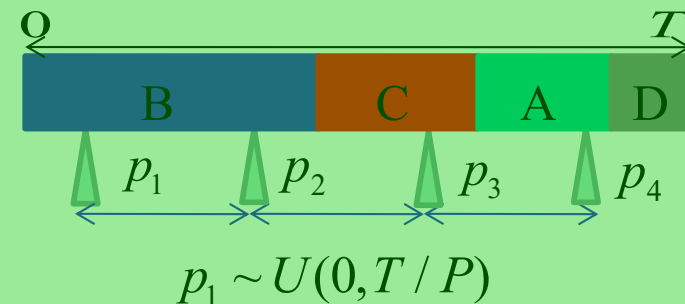
Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

- Amostragem estocástica uniforme (*uniform stochastic sampling*) – define-se uma linha de comprimento igual ao total das aptidões dos indivíduos da população. É gerado uma única posição inicial de valor menor que o passo T/P e depois os progenitores são selecionados em passos de igual tamanho ao longo da linha.

@selectionstochunif

$$T = \sum_{i=1}^P F(\mathbf{x})$$



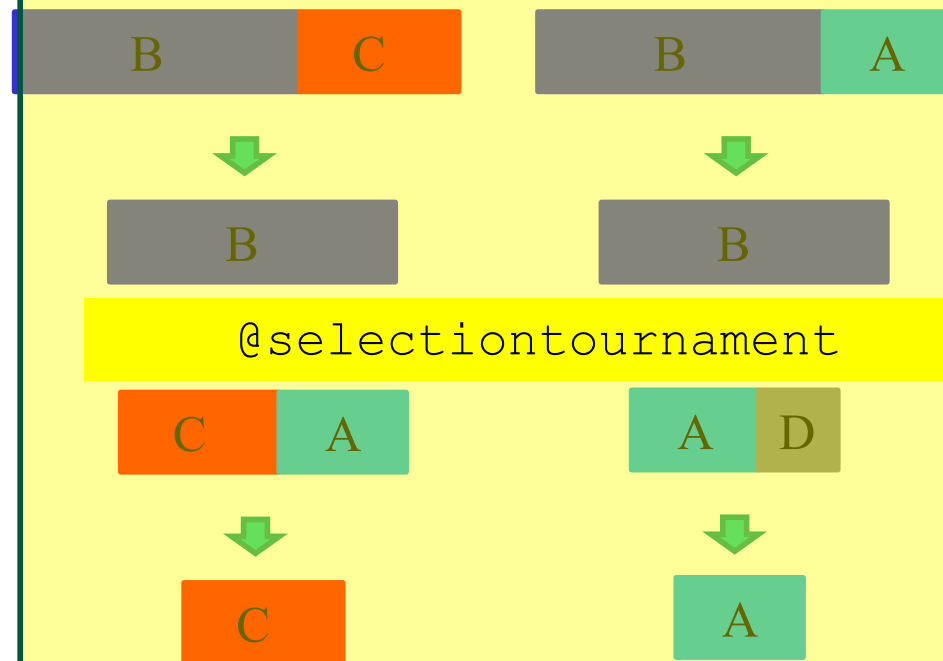


Seleção parental - SelectionFcn

Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. **Seleção parental**
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

- torneio (*tournament*) – os progenitores são escolhidos, selecionado aleatoriamente um determinado número de indivíduos da população para torneio. O indivíduo mais apto de cada torneio é escolhido para progenitor.



• ...



Configuração do ga

Universidade do Minho

`'CrossoverFcn'` – especifica o tipo de cruzamento

- `@crossoverscattered` – cruzamento uniforme (valor por defeito).
- `@crossoveringlepoint` – cruzamento em 1 ponto.
- `@crossovertwopoint` – cruzamento em 2 pontos.
- `@crossoverintermediate` – cruzamento intermédio. Permite especificar pesos (Ratio). Por defeito, Ratio=1.
- `@crossoverheuristic` – cruzamento intermédio enviesado para o melhor progenitor. É possível, especificar um peso (Ratio). Por defeito, Ratio=1.2.
- `@crossoverarithmetic` – cruzamento aritmético.
- `Custom` – permite definir o operador de cruzamento.

MatLab

Para passar a utilizar cruzamento em dois pontos, fazer

```
options = optimoptions(@ga, options, 'CrossoverFcn', @crossovertwopoint );  
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Recombinação - CrossoverFcn

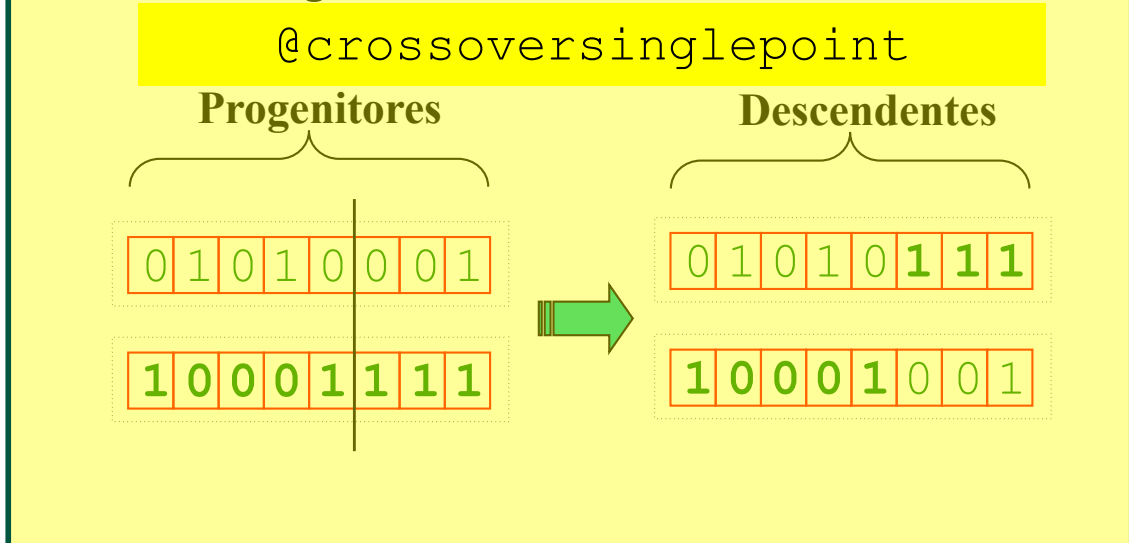
Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. **Recombinação**
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Pares de progenitores são re combinados para gerar descendentes. O tipo de recombinação depende da representação usada e pode ser aplicada com uma determinada probabilidade.

Exemplos de operadores de recombinação para indivíduos com representação binária:

- um ponto (*one-point crossover*) – Um ponto de crossover gerado aleatoriamente entre 1 e $L-1$.



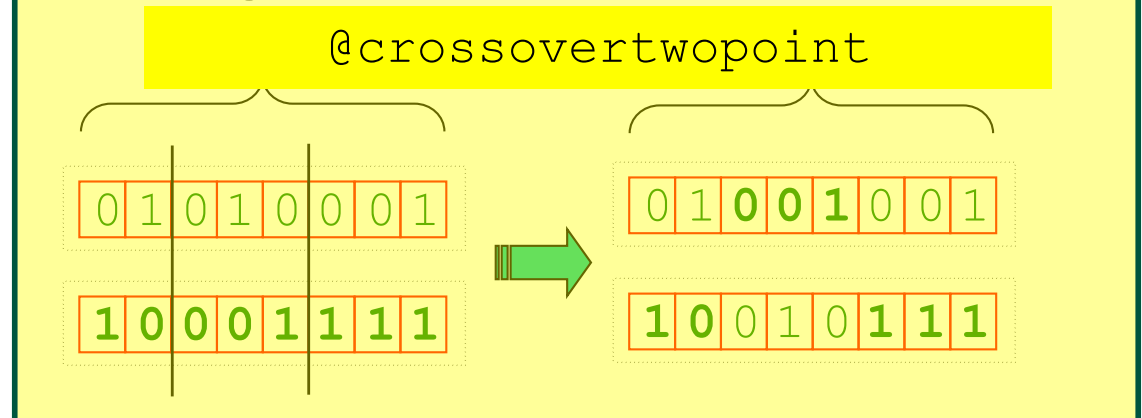


Recombinação - CrossoverFcn

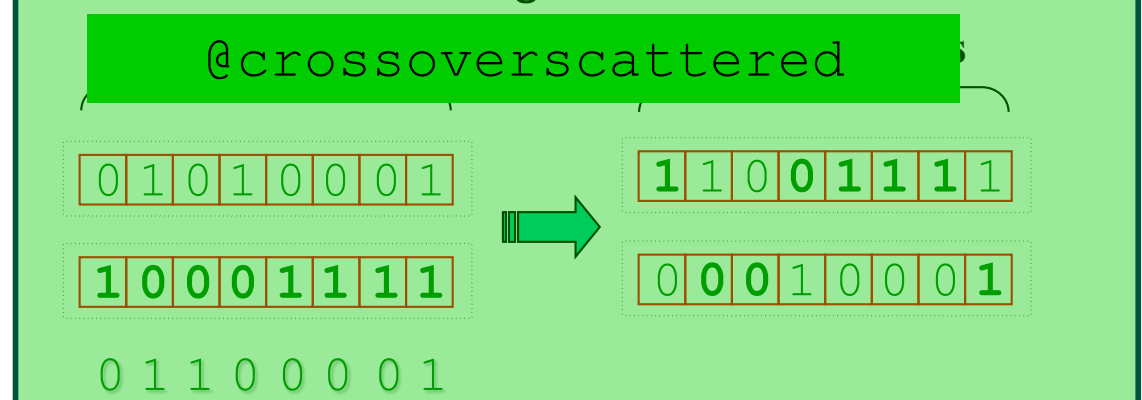
Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. **Recombinação**
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

- dois pontos (*two-point crossover*) – dois pontos de crossover gerados aleatoriamente entre 1 e $L-1$.



- disperso (*scattered crossover*) - Uma máscara binária de tamanho L é gerada aleatoriamente.



...



Recombinação - CrossoverFcn

Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Em geral, os operadores de recombinação para representação binária podem ser aplicados a indivíduos com representação em vetor de números reais ou vetor de números inteiros.

Exemplos de operadores de recombinação específicos para indivíduos com representação em vetor de números reais:

- aritmético (*arithmetic crossover*)

$$x_i^{D_1} = \text{@crossoverarithmetic}$$

- intermédio (*intermediate crossover*)

$$x_i^{D_1} = P_1 + r(P_2 - P_1) \quad \text{onde } r \sim U(0,1)$$

`@crossoverintermediate`



Recombinação - CrossoverFcn

Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. Mutação
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

- binário simulado (*SBX - simulated binary crossover*)
$$x_i^{D_1} = 0.5 \left((1 + \beta_i) x_i^{P_1} + (1 - \beta_i) x_i^{P_2} \right)$$
$$x_i^{D_2} = 0.5 \left((1 - \beta_i) x_i^{P_1} + (1 + \beta_i) x_i^{P_2} \right) \quad \text{para } i = 1, \dots, n$$
$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c + 1}} & \text{se } u_i \leq 0.5 \\ \left(\frac{1}{2(1 - u_i)} \right)^{\frac{1}{\eta_c + 1}} & \text{caso contrário} \end{cases} \quad \text{com } u_i \sim U(0,1)$$

\text{'custom'}
- linear (*linear crossover*)
$$x_i^{D_1} = 0.5(x_i^{P_1} + x_i^{P_2})$$
$$x_i^{D_2} = 1.5x_i^{P_1} - 0.5x_i^{P_2} \quad \text{para } i = 1, \dots, n$$
$$x_i^{D_3} = -0.5x_i^{P_1} + 1.5x_i^{P_2}$$

\text{'custom'}
- ...



Configuração do ga

Universidade do Minho

`'MutationFcn'` – especifica o tipo de mutação

- `@mutationgaussian` — mutação gaussiana para problemas sem restrições (valor por defeito). Utiliza uma regra de adaptação baseada numa redução linear. É possível especificar o valor inicial (Scale) e taxa de redução (Shrink). Por defeito, Scale=1 e Shrink=1. É possível utilizar a mutação gaussiana de forma isotrópica e não isotrópica. Não implementa autoadaptação (coevolução). Não é adequada a problemas com restrições pois não garante admissibilidade.
- `@mutationuniform` — mutação uniforme (por defeito, probabilidade de mutação é 0.01)
- `@mutationadaptfeasible` — mutação adaptativa (valor por defeito para problemas com restrições). Adequada para problemas com restrições, pois garante admissibilidade.
- `Custom` — permite definir o operador de mutação.

MatLab

Para passar a utilizar a mutação gaussiana com Scale=1 e Shrink=0.5, fazer

```
options = optimoptions(@ga,options, 'MutationFcn', {@ mutationgaussian,1,0.5 });  
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Mutação - MutationFcn

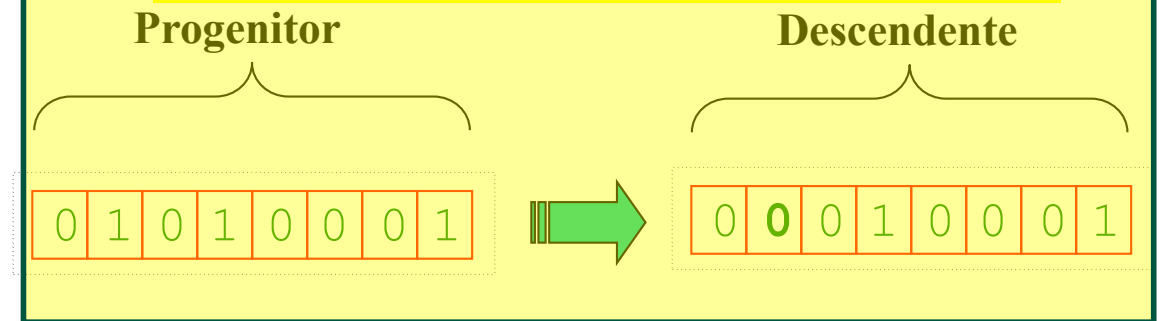
Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. **Mutação**
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Cada progenitor é mutado para gerar um descendente. O tipo de mutação depende da representação usada e pode ser aplicada com uma determinada probabilidade.

Exemplos de operadores de mutação para indivíduos com representação binária:

- uniforme (*uniform mutation*) – em cada posição é gerado um número aleatório uniforme $u \sim U(0,1)$ e se $u < p$, a posição é mutada (em geral $p = 1/L$)
@mutationuniform



• ...



Mutação - MutationFcn

Universidade do Minho

1. Criação da população inicial
2. Avaliação dos indivíduos
3. Enquanto (CP falso) fazer
4. Seleção parental
5. Recombinação
6. **Mutação**
7. Avaliação dos indivíduos
8. Seleção ambiental
9. FimEnquanto

Em geral, os operadores de mutação para representação binária podem ser aplicados a indivíduos com representação em vetor de números reais ou vetor de números inteiros.

Exemplos de operadores de mutação específicos para indivíduos com representação em vetor de números reais:

- aleatória (*random mutation*)

$$x_i^D = x_i^P + u_i(\beta_i - \alpha_i) \quad \text{para } i = 1, \dots, n$$

com $u_i \sim U(0,1)$

onde α_i e β_i são os limites simples da variável i



Mutação - MutationFcn

Universidade do Minho

1. Criação da população inicial

2. Avaliação dos indivíduos

3. Enquanto (CP falso) fazer

4. Seleção parental

5. Recombinação

6. **Mutação**

7. Avaliação dos indivíduos

8. Seleção ambiental

9. FimEnquanto

- gaussiana (*gaussian mutation*)

$$x_i^D = x_i^P + z_i \quad \text{para } i=1,\dots,n$$

@mutationgaussian

- polinomial (*polynomial mutation*)

$$x_i^D = x_i^P + (\beta_i - \alpha_i) \delta_i, n$$

onde α_i e β_i são os limites simples da variável i

$$\delta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_m+1}} - 1 & \text{se } u_i < 0.5 \\ 1 - (2(1-u_i))^{\frac{1}{\eta_m+1}} & \text{caso contrário} \end{cases} \quad \text{com } u_i \sim U(0,1)$$

- ...



...comando optimoptions

Universidade do Minho

```
Display: [ 'off' | 'iter' | 'diagnose' | {'final'} ]
OutputFcns: [ function_handle | {} ]
PlotFcns: [ function_handle | @gaplotbestf | @gaplotbestindiv | @gaplotdistance |
            @gaplotexpectation | @gaplotgenealogy | @gaplotselection | @gaplotrange |
            @gaplotscorediversity | @gaplotscores | @gaplotstopping |
            @gaplotmaxconstr | @gaplotrankhist | @gaplotpareto | @gaplotspread |
            @gaplotparetodistance | {} ]
PlotInterval: [ positive scalar | {1} ]

Vectorized: [ 'on' | {'off'} ]

UseParallel: [ logical scalar | true | {false} ]
```

MatLab

Para alterar uma opção na estrutura `options`, mantendo as restantes opções, e executar o ga fazer:

```
options = optimoptions(@ga)
options = optimoptions(@ga, options, PlotFcn', @gaplotbestf)
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)

options = optimoptions(@ga, options, PlotFcn', {@gaplotbestf,@gaplotrange})
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Reprodução de resultados

Universidade do Minho

Os Algoritmos Genéticos são estocásticos, logo diferentes execuções levam a resultados diferentes. O comando `ga` usa os geradores de números aleatórios `rand` (dist. uniforme) e `randn` (dist. normal). O estado dos geradores após uma execução do `ga` é devolvido em `output.randstate` e `output.randnstate`.

Logo, é possível reproduzir os resultados, fazendo antes da nova execução do comando `ga`:

```
rng default
```

MatLab

Invoque duas vezes seguida o `ga`:

```
[x1,fval1,exitflag1,output1] = ga(@rastrigins,2)
rng default
[x2,fval2,exitflag2,output2] = ga(@rastrigins,2)
```

Verifique que os resultados obtidos são diferentes. Execute agora os comandos:

```
rng default
[x3,fval3,exitflag3,output3] = ga(@rastrigins,2)
```

Verifique que os resultados obtidos agora são iguais à última das duas execuções anteriores.



Execução do ga com restrições de limite

Universidade do Minho

É possível definir restrições de limite simples nas variáveis, passando como argumentos de entrada matrizes com esses limites (LB e UB):

```
[x,fval] = ga(@fitnessfun, nvars, [],[],[],[],LB,UB,[],options)
```

MatLab

Resolva o problema incluindo agora as seguintes restrições de limite simples: $-5 \leq x_1 \leq 5$ e $1 \leq x_2 \leq 5$

```
LB=[-5 1]
```

```
UB=[5 5]
```

```
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],LB,UB,[],options)
```

Comente o resultado obtido.

Altere o tipo de mutação para @mutationadaptfeasible e repita a procura.

```
options = optimoptions(@ga,options, 'MutationFcn', @mutationadaptfeasible);
```

```
LB=[-5 1]
```

```
UB=[5 5]
```

```
[x,fval,exitflag,output] = ga(@rastrigins,2,[],[],[],[],LB,UB,[],options)
```




Exercícios Propostos

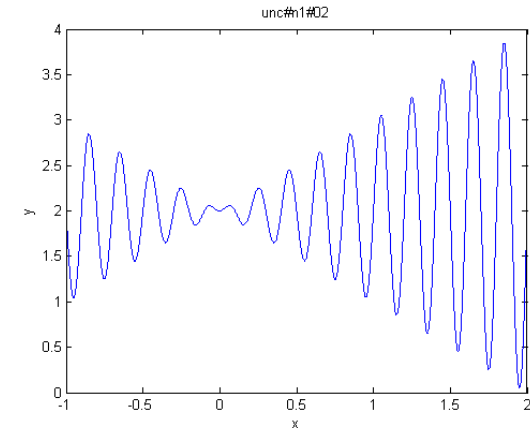
Universidade do Minho

MatLab

1. Utilizando o comando `ga` encontre uma aproximação para o minimizante global do seguinte problema:

$$\min f(x) = x \sin(10\pi x) + 2.0$$

$$s.a. \quad -1 \leq x \leq 2$$



2. Utilizando o comando `ga` encontre uma aproximação para o minimizante global do seguinte problema definido para n variáveis (Schwefel):

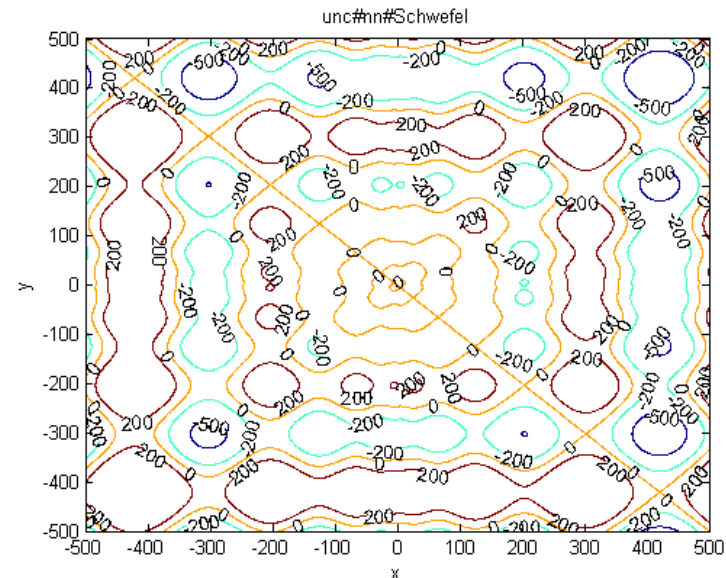
$$\min f(\mathbf{x}) = \sum_{i=1}^n -x_i \sin(\sqrt{|x_i|})$$

$$s.a. \quad -500 \leq x_i \leq 500 \quad (i = 1, \dots, n)$$

Resolva o exercício, considerando:

a) $n=2$

b) $n=5$





...comando gaoptimset

Universidade do Minho

...

```
InitialPopulation: [ matrix          | {[[]]} ]  
InitialScores: [ column vector     | {[[]]} ]
```

```
NonlinConAlgorithm: [ 'penalty' | {'auglag'} ]  
InitialPenalty: [ positive scalar | {10} ]  
PenaltyFactor: [ positive scalar | {100} ]
```

• Tipo de
Tratamento
de restrições

```
CreationFcn: [ function_handle | @gacreationuniform |  
@gacreationlinearfeasible ]
```

```
FitnessScalingFcn: [ function_handle | @fitscalingshiftlinear | @fitscalingprop |  
@fitscalingtop | {@fitscalingrank} ]  
SelectionFcn: [ function_handle | @selectionremainder | @selectionuniform |  
@selectionroulette | @selectiontournament |  
@selectionstochunif ]  
CrossoverFcn: [ function_handle | @crossoverheuristic | @crossoverintermediate  
|  
@crossoveringlepoint | @crossovertwopoint |  
@crossoverarithmetic |  
@crossoverscattered ]  
MutationFcn: [ function_handle | @mutationuniform | @mutationadaptfeasible |  
@mutationgaussian ]
```

```
DistanceMeasureFcn: [ function_handle | {@distancecrowding} ]
```

```
HybridFcn: [ @fminsearch | @patternsearch | @fminunc | @fmincon | {[[]]} ] ...
```



Execução do ga com restrições

Universidade do Minho

No tratamento de restrições não lineares, no comando `ga` é utilizada uma formulação baseada na função lagrangeana aumentada.

```
[x,fval] = ga(@fitnessfun,nvars,A,b,Aeq,beq,LB,UB,NONLCON,options)
```

MatLab

Resolva o problema com restrições:

```
LB=[13 0]
```

```
UB=[100 100]
```

$$\min f(x_1, x_2) = (x_1 - 10)^3 + (x_2 - 20)^3$$

$$s.a \quad 13 \leq x_1 \leq 100, \quad 0 \leq x_2 \leq 100$$

$$-(x_1 - 5)^2 - (x_2 - 5)^2 + 100 \leq 0$$

$$(x_1 - 6)^2 + (x_2 - 5)^2 - 82.81 \leq 0$$

```
[x,fval,exitflag,output] = ga(@g06,2,[],[],[],[],LB,UB,@g06_constr,options)
```

Verifique se a solução obtida é admissível.



Execução do ga com variáveis inteiras

Universidade do Minho

No comando `ga` é possível indicar que algumas (ou todas) as variáveis devem ser inteiras.

```
[x,fval] = ga(@fitnessfun,nvars,A,b,Aeq,beq,LB,UB,NONLCON,intcon)
```

MatLab

Resolva o problema considerando x_1 inteiro

```
LB=[13 0]  
UB=[100 100]  
intcon=[1]
```

```
[x,fval,exitflag,output] = ga(@g06,2,[],[],[],[],LB,UB,@g06_constr,intcon)
```

$$\min f(x_1, x_2) = (x_1 - 10)^3 + (x_2 - 20)^3$$

$$s.a \quad 13 \leq x_1 \leq 100, \quad 0 \leq x_2 \leq 100$$

$$-(x_1 - 5)^2 - (x_2 - 5)^2 + 100 \leq 0$$

$$(x_1 - 6)^2 + (x_2 - 5)^2 - 82.81 \leq 0$$

Resolva agora o problema considerando x_1 e x_2 inteiros

```
LB=[13 0]  
UB=[100 100]  
intcon=[1,2]
```

```
[x,fval,exitflag,output] = ga(@g06,2,[],[],[],[],LB,UB,@g06_constr,intcon)
```



...comando gaoptimset

Universidade do Minho

```
...
InitialPopulation: [ matrix          | {} ]
InitialScores: [ column vector      | {} ]

NonlinConAlgorithm: [ 'penalty' | {'auglag'} ]
InitialPenalty: [ positive scalar | {10} ]
PenaltyFactor: [ positive scalar | {100} ]

CreationFcn: [ function_handle | @gacreationuniform |
@gacreationlinearfeasible ]

FitnessScalingFcn: [ function_handle | @fitscalingshiftlinear | @fitscalingprop |
@fitscalingtop | {@fitscalingrank} ]
SelectionFcn: [ function_handle | @selectionremainder | @selectionuniform |
@selectionroulette | @selectiontournament |
@selectionstochunif ]
CrossoverFcn: [ function_handle | @crossoverheuristic | @crossoverintermediate
|
@crossoversinglepoint | @crossovertwopoint |
@crossoverarithmetic |
@crossoversscattered ]
MutationFcn: [ function_handle | @mutationuniform | @mutationadaptfeasible |
@mutationgaussian ]

DistanceMeasureFcn: [ function handle | {@distancecrowding} ]
HybridFcn: [ @fminsearch | @patternsearch | @fminunc | @fmincon | {} ] ...
```

• Hibridização com métodos
de procura local



Execução do ga com hibridização

Universidade do Minho

É possível aplicar um método de procura local após a procura do ga (hibridização).

MatLab

Encontre o minimizante da função Rastrigins usando o ga com hibridização.

$$f(x_1, x_2) = 20 + x_1^2 + x_2^2 - 10(\cos(2\pi x_1) + \cos(2\pi x_2))$$

```
% sem hibridização
```

```
[x,fval] = ga(@rastrigins,2)
```

```
% hibridização com fminsearch
```

```
options = optimoptions(@ga,'HybridFcn', @fminsearch);
```

```
[x,fval] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```

```
% hibridização com fminunc
```

```
options = optimoptions(@ga,'HybridFcn', @fminunc);
```

```
[x,fval] = ga(@rastrigins,2,[],[],[],[],[],[],[],options)
```



Exercícios Propostos

Universidade do Minho

MatLab

1. Utilizando o comando `ga` encontre uma aproximação para o minimizante global do seguinte problema:

$$\begin{aligned} \min f(x_1, x_2) &= x_1^2 + (x_2 - 1)^2 \\ \text{s.a.} \quad &-1 \leq x_1 \leq 1, \quad -1 \leq x_2 \leq 1 \\ &x_2 - x_1^2 = 0 \end{aligned}$$

2. Utilizando o comando `ga` encontre uma aproximação para o minimizante global do seguinte problema:

$$\begin{aligned} \max f(x, y) &= \frac{\sin^3(2\pi x) \sin(2\pi y)}{x^3(x+y)} \\ \text{s.a.} \quad &0 \leq x \leq 10 \text{ e } 0 \leq y \leq 10 \\ &x^2 - y + 1 \leq 0 \\ &1 - x + (y - 4)^2 \leq 0 \end{aligned}$$